

Phase 1 — Python, Git & Virtual Environments in WSL

All hands-on labs in this curriculum require execution within **WSL (Windows Subsystem for Linux)** on corporate developer laptops. Running code inside WSL ensures maximum compatibility with linux-native LangGraph persistence libraries while maintaining isolation from host Windows registry controls.

1.1 WSL Environment Requirements

Ensure you have WSL2 enabled on your Windows laptop. In the WSL shell (Ubuntu/Debian distribution), verify that Python 3.10 or 3.11 is active:

```
python3 --version
```

1.2 Virtual Environment Initialization inside WSL

Isolating project dependencies within your WSL environment is critical to avoid system-level package conflicts. Navigate to your project directory inside the WSL filesystem and run:

```
# Create a virtual environment named .venv inside your workspace
python3 -m venv .venv

# Activate the environment in the WSL terminal
source .venv/bin/activate
```

1.3 Mount File Sharing (Windows to WSL)

To access local files stored on your Windows host filesystem (e.g. downloaded certificates, custom data files), use the mount point `/mnt/c/` in WSL. For example, to copy a file from your Windows Documents directory, run:

```
cp /mnt/c/Users/YourUsername/Documents/config.json ~/workspace/
```

1.4 Safe Git Configuration in WSL

If you are pulling curriculum templates or exercises from an internal enterprise Git server behind a secure proxy, SSL checks may fail. You can bypass global SSL certificate verification inside WSL safely for the training scope by running:

```
git config --global http.sslVerify false
```

Phase 2 — Network Proxies & SSL Certificates Setup in WSL

Corporate developer laptops require proxy forwarding rules and certificate paths to be mapped inside the WSL environment to bypass connection timeouts.

2.1 Configuring WSL Terminal Proxy Variables

Export the HTTP and HTTPS proxy variables in your WSL terminal before running python scripts or package installations:

```
export HTTP_PROXY="http://proxy.yourcorporate.com:8080"
export HTTPS_PROXY="http://proxy.yourcorporate.com:8080"
export NO_PROXY="localhost,127.0.0.1,11434"
```

2.2 Persisting Proxy Rules across WSL Sessions

To prevent entering the proxy variables manually every time you launch your WSL shell, append the environment exports to your user's shell configuration file:

```
echo 'export HTTP_PROXY="http://proxy.yourcorporate.com:8080"' >> ~/.bashrc
echo 'export HTTPS_PROXY="http://proxy.yourcorporate.com:8080"' >> ~/.bashrc
echo 'export NO_PROXY="localhost,127.0.0.1,11434"' >> ~/.bashrc
source ~/.bashrc
```

2.3 Handling SSL Certificate Verification Failures in WSL

If your corporate network performs SSL decryption and inspection on outbound web traffic, Python libraries will throw SSL validation errors. To fix this, access your corporate root CA PEM certificate file (e.g. from your Windows mount) and configure WSL to route python connections through it:

```
# Set env variables pointing to your corporate root certificate path
export REQUESTS_CA_BUNDLE="/mnt/c/Certificates/corporate-ca-bundle.pem"
export SSL_CERT_FILE="/mnt/c/Certificates/corporate-ca-bundle.pem"
```

Phase 3 — Offline Package Caching & Local Installation

For corporate developer laptops that are completely air-gapped or restricted from querying public PyPI registers directly, follow this caching installation process.

3.1 Pre-downloading Packages (On an Unrestricted Machine)

On a machine with full internet access, run the download command to cache all required training packages into a directory named `pip\_cache`:

```
pip download -r requirements.txt -d ./pip_cache
```

3.2 Transferring Cache Directory to WSL

Move the `pip\_cache` directory to a folder on your Windows host (e.g. `C:\Temp\pip\_cache`). Within WSL, verify that the folder is visible in the mount path:

```
ls -la /mnt/c/Temp/pip_cache/
```

3.3 Executing Offline Installation in WSL

Once the cache directory is verified in WSL, run the package installer in offline mode. This installs LangChain, LangGraph, and other packages without making external network calls:

```
pip install --no-index --find-links=/mnt/c/Temp/pip_cache/ -r requirements.txt
```

Phase 4 — Paid APIs Setup (OpenAI / Anthropic) & Local Mocks

To run the course exercises using cloud models, developers must configure WSL to securely export paid API keys (OpenAI or Anthropic) and route outbound API requests through the corporate proxy.

4.1 API Key Configuration & Shell Persistence in WSL

Export the API keys in your WSL terminal session. To avoid entering these keys on every new terminal session, append them to your user's shell configuration profile:

```
# Add API Keys to your WSL bash profile
echo 'export OPENAI_API_KEY="sk-proj-yourOpenAiKeyHere..." >> ~/.bashrc
echo 'export ANTHROPIC_API_KEY="sk-ant-api03-yourAnthropicKeyHere..." >> ~/.bashrc
source ~/.bashrc
```

4.2 Proxy Routing for External API Endpoints

Because the laptop is inside a restricted corporate network, all outbound requests to *api.openai.com* or *api.anthropic.com* must pass through the HTTP/HTTPS proxy configured in Phase 2. Verify that network routing and certificate verification succeed by running:

```
# Test connectivity to OpenAI endpoint through your proxy
curl -I https://api.openai.com/v1/models -H "Authorization: Bearer $OPENAI_API_KEY"

# Test connectivity to Anthropic endpoint through your proxy
curl -I https://api.anthropic.com/v1/messages --header "x-api-key: $ANTHROPIC_API_KEY" --header "anthropic-version: 2023-06-01"
```

4.3 LangChain Model & Embeddings Initialization

Configure LangChain to import and initialize the appropriate model and embedding instance, which automatically picks up the exported keys from your WSL environment shell variables:

```
# 1. For OpenAI (Requires langchain-openai package)
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
llm = ChatOpenAI(model="gpt-4o", temperature=0)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# 2. For Anthropic (Requires langchain-anthropic package)
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model="claude-3-5-sonnet-20240620", temperature=0)
```

4.4 Local Mock CRM REST Server in WSL

For integration labs (such as customer data lookups), write and run this mock CRM daemon on port 8000 inside WSL to mimic external billing REST APIs:

```
import http.server
import json

class MockCRM(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps({"id": 1, "name": "Rohan Sharma", "balance": 150.00}).encode())

server = http.server.HTTPServer(('localhost', 8000), MockCRM)
server.serve_forever()
```

Phase 5 — WSL Day 1 & Day 2 Lab Verification Suite

Use these scripts inside WSL to verify package bindings and model response execution. Ensure your virtual environment is active before running them.

5.1 Day 1 Verification: WSL RAG pipeline. File name: `verify\_day1.py`

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

# 1. Initialize OpenAI model and embeddings (automatically resolved from environment keys)
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 2. Build local FAISS vector store
texts = [
    "Corporate policy: All travel requests must be approved 2 weeks in advance.",
    "Expense policy: Food expenses are capped at $50 per day."
]
vector_store = FAISS.from_texts(texts, embeddings)
retriever = vector_store.as_retriever()

# 3. Create RAG chain
prompt = ChatPromptTemplate.from_template("Context: {context}\nQuestion: {question}\nAnswer:")
chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

# 4. Invoke
response = chain.invoke("What is the food expense cap?")
print("Day 1 Verification Output:\n", response)
```

5.2 Day 2 Verification: WSL ReAct Tool Bindings. File name: `verify\_day2.py`

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_customer_balance(account_id: str) -> float:
    """Query the local mock CRM API to retrieve current balance."""
    return 150.00

# Initialize OpenAI model and bind tools
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
llm_with_tools = llm.bind_tools([get_customer_balance])

res = llm_with_tools.invoke("Check balance details for account user ACC-101")
print("Day 2 Verification Output (Tool Calls):\n", res.tool_calls)
```

Phase 5 — WSL Day 3 & Day 4 Lab Verification Suite

Execute these workflows inside WSL to verify complex state compilation and multi-agent coordination.

5.3 Day 3 Verification: WSL LangGraph StateGraph. File name: `verify\_day3.py`

```
from typing import TypedDict, List
from langgraph.graph import StateGraph, END

class AgentState(TypedDict):
    messages: List[str]
    current_value: int

def node_addition(state: AgentState):
    val = state.get("current_value", 0) + 10
    return {"messages": ["Added 10"], "current_value": val}

def node_multiplier(state: AgentState):
    val = state.get("current_value", 0) * 2
    return {"messages": ["Multiplied by 2"], "current_value": val}

workflow = StateGraph(AgentState)
workflow.add_node("adder", node_addition)
workflow.add_node("multiplier", node_multiplier)
workflow.set_entry_point("adder")
workflow.add_edge("adder", "multiplier")
workflow.add_edge("multiplier", END)

app = workflow.compile()
result = app.invoke({"messages": ["Start"], "current_value": 5})
print("Day 3 Verification Output:\n", result)
```

5.4 Day 4 Verification: Multi-Agent supervisor routing. File name: `verify_day4.py`

```
from typing import Literal, TypedDict, List
from langgraph.graph import StateGraph, END

class TeamState(TypedDict):
    task: str
    active_worker: str
    log: List[str]

def supervisor(state: TeamState):
    next_worker = "analyst" if "data" in state["task"] else "writer"
    return {"active_worker": next_worker, "log": [f"Supervisor routed to {next_worker}"]}

def analyst_worker(state: TeamState):
    return {"log": ["Analyst processed data records"], "active_worker": "END"}

def writer_worker(state: TeamState):
    return {"log": ["Writer generated output text"], "active_worker": "END"}

builder = StateGraph(TeamState)
builder.add_node("supervisor", supervisor)
builder.add_node("analyst", analyst_worker)
builder.add_node("writer", writer_worker)
builder.set_entry_point("supervisor")

builder.add_conditional_edges("supervisor", lambda s: s["active_worker"])
builder.add_edge("analyst", END)
builder.add_edge("writer", END)

team = builder.compile()
out = team.invoke({"task": "analyze sales data", "active_worker": "", "log": []})
print("Day 4 Verification Output:\n", out["log"])
```


WSL Telemetry Verification & Troubleshooting FAQ

5.5 Day 5 Verification: Telemetry connection. File name: `verify\_day5.py`

```
import os
import requests

# Set verification key values for checking trace endpoint routes
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "ls__testkey_12345"
os.environ["LANGCHAIN_ENDPOINT"] = "https://api.smith.langchain.com"

print("Checking LangSmith connection from WSL...")
try:
    res = requests.get("https://api.smith.langchain.com/status", timeout=5)
    print("LangSmith Connection Test Status:", res.status_code)
except Exception as e:
    print("LangSmith connection bypass or proxy needed. Connection error details:", e)
```

5.6 Troubleshooting WSL Connectivity Issues

Q: I run verify_day1.py or verify_day2.py and get connection timeout / authentication errors.

A: Ensure your API keys are valid and exported inside the active WSL terminal session. If behind a corporate proxy, check that your HTTP_PROXY and HTTPS_PROXY shell variables are set (Section 2.1) and that your custom root CA certificate path (REQUESTS_CA_BUNDLE) is loaded (Section 2.3) so python requests can establish secure SSL sessions.

Q: I run verify_day3.py or verify_day4.py but get module not found errors for langgraph.

A: Your active virtual environment has not been activated, or packages were installed on the Windows host instead of inside the WSL shell. Run `source .venv/bin/activate` inside WSL, then verify the installations using `pip list`.

Q: My corporate proxy blocks pip requests from within WSL.

A: WSL does not inherit host environment proxy values automatically. Ensure proxy lines are explicitly exported in WSL, or added to your WSL `~/.bashrc` file (Section 2.2).

Q: Tracing to LangSmith or Langfuse fails from WSL.

A: Ensure `api.smith.langchain.com` is added to your terminal `HTTPS\_PROXY` values inside WSL. If tracing endpoint requests are dropped entirely by your corporate proxy, you can setup a local self-hosted Langfuse Docker container on your Windows machine and map WSL telemetry outputs to it.